1 Web Essentials: Clients, Servers, Communication and Markup Languages

- Elements of WWW
- Basic Internet Protocols
  - Intro
  - Several Internet protocols
  - Working of TCP/IP
  - TCP vs UDP
  - HTTP
  - DNS
- Client-server Model
- Introduction to Markup Languages

Readings
- Web Technologies: A Computer Science Perspective (Chapter 1)

1.1 Elements of WWW

Server
- The software that distributes the information
- Machine where the information and software reside
- Example: web server
-

Client
- The software that resides on the remote machine, communicates with the server, fetches the information, processes it, and then displays it on the remote machine
  - initiates contact with server (- speaks first)
  - typically requests service from server
  - Typically implemented in browser

Web server
- Software that delivers Web pages and other documents to browsers using the HTTP protocol

Web Page
- A document or resource of information that is suitable for the World Wide Web and can be accessed through a web browser.

Website
- A collection of pages on the World Wide Web that are accessible from the same URL and typically residing on the same server.

URL
- Uniform Resource Locator, the unique address which identifies a resource on the Internet for routing purposes

1.2 Basic Internet Protocols

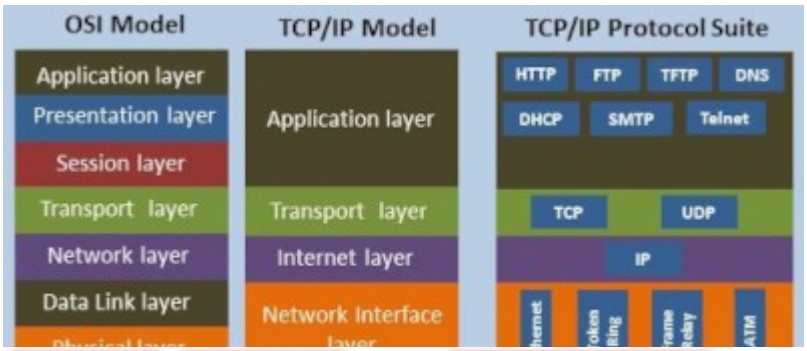
1.2.1 Introduction

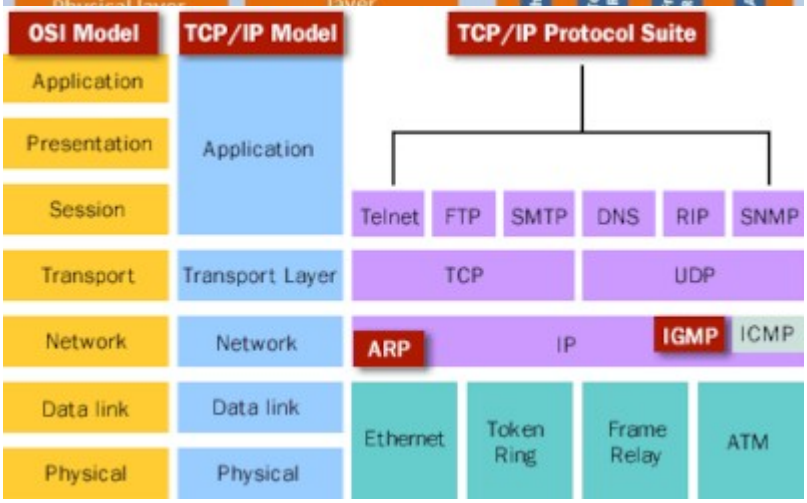
Protocol
- Detailed specification of how communication between two computers will be carried out in order to serve some purpose

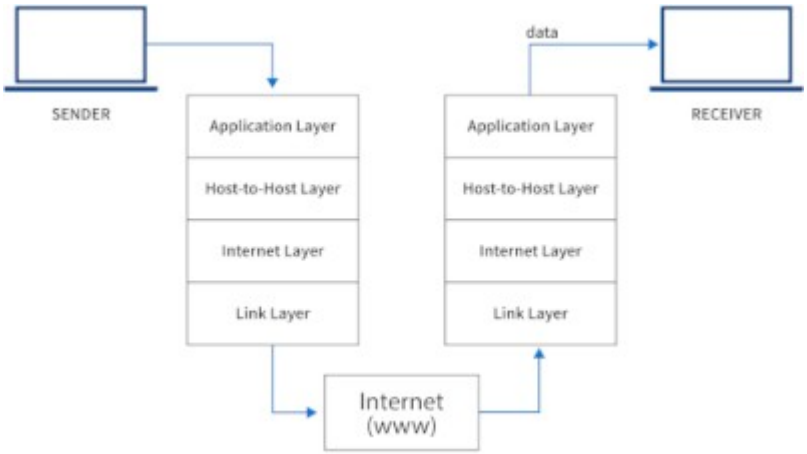
- Determines several things including
  - The error checking required
  - Data compression method used
  - The way the end of a message is signalled
  - The way the device indicates that it has received the message

1.2.2 Several internet protocols

- File Transfer Protocol (FTP) – file transfer
- Simple Mail Transfer Protocol (SMTP) – email communication
- TCP/IP (Transmission Control Protocol/Internet Protocol) communication protocol
  - TCP/IP was designed to be used for host-to-host communication both within local area networks (that is, networks of computers that are typically in close proximity to one another, such as within a building) and between networks
  - Comprises 2 protocols
    - Transmission control protocol (TCP)
      - Controls disassembly of message into packets at the origin
      - Reassembling of messages at the destination
    - Internet protocol (IP)
      - Specifies the addressing details for each packet
      - Each packet is labelled with its origin and destination IP addresses
- Etc.







1.2.3 Working of TCP/IP

https://www.avg.com/en/signal/what-is-tcp-ip
https://www.educba.com/what-is-tcp-ip/
https://www.cloudns.net/blog/tcp-transmission-control-protocol-what-is-it-and-how-does-it-work/
https://www.lifesize.com/blog/tcp-vs-udp/

1.2.4 TCP (Transmission Control Protocol) vs UDP (User Datagram Protocol)

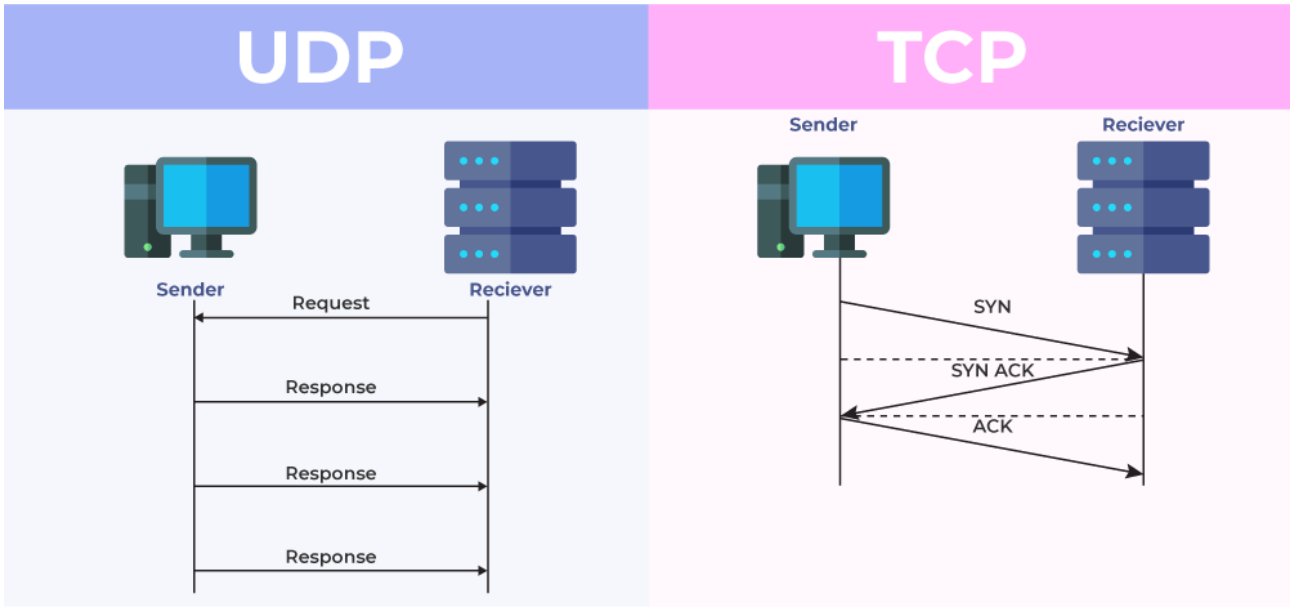
Transport layer actually comprises two protocols – TCP and UDP – and only one of them is used to transport data, depending on the requirements

TCP

- Connection-oriented: once a connection has been established, data can be transmitted in two directions
- Has built-in systems to check for errors and to guarantee data will be delivered in the order it was sent, making it the perfect protocol for transferring information like still images, data files, and web pages.

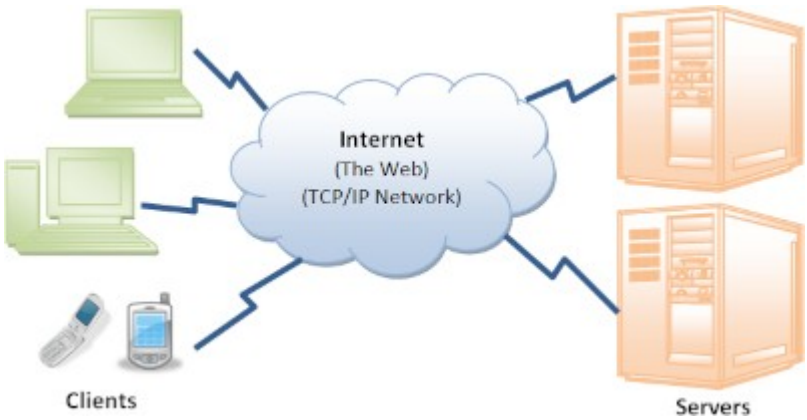
UDP

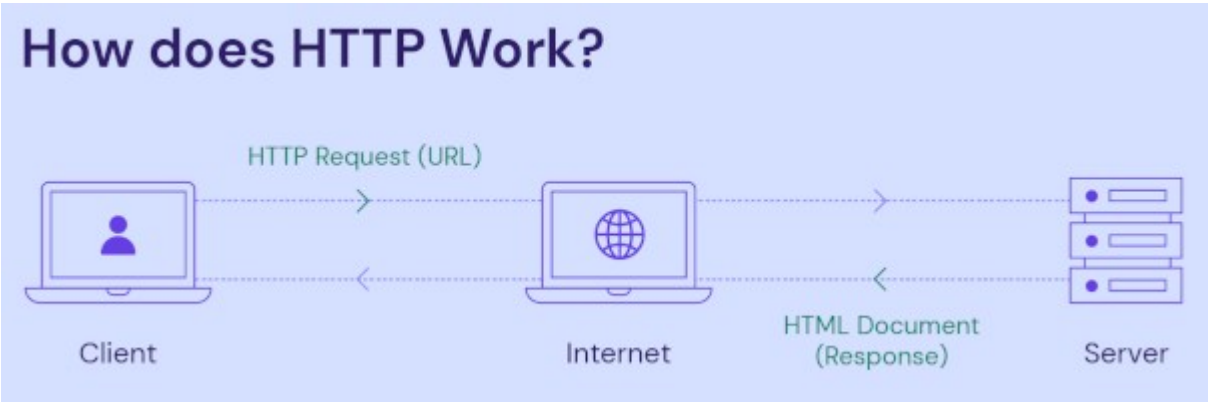
- Simpler, connectionless protocol
- No error-checking and recovery services
- No overhead for opening a connection, maintaining a connection, or terminating a connection; data is continuously sent to the recipient, whether or not they receive it
- Preferred for real-time communications like broadcast

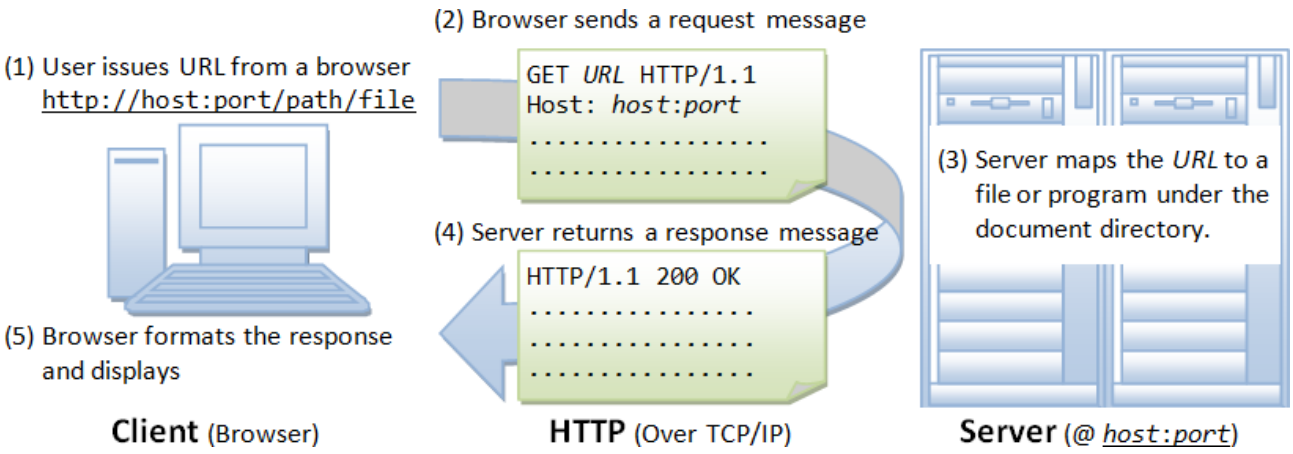


1.2.5 Hypertext Transport Protocol (HTTP)

- Protocol used by the Internet to transfer hypertext documents (hypertext requests and information) between servers and browsers
  - Hypertext is text, displayed on a computer, with references (hyperlinks) to other text that the reader can immediately follow, usually by clicking a mouse button
  - HTTP is behind every request for a web document, every click of a hypertext link, and every submission of a form
- Works on top of TCP/IP suite of protocols
- Specifies how clients request data, and how servers respond to these requests







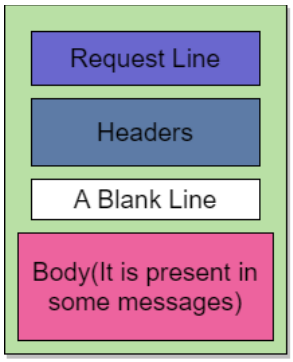
Working

- Uses client-server model (Web browser is the client, web server the server)
- Whenever a client requests some information (say clicks on a hyperlink)
  - The browser sends a request message to the HTTP server for the requested objects
  - Then
    - HTTP opens a connection between the client and server through TCP
    - HTTP sends a request to the server which collects the requested data
    - HTTP sends the response with the objects back to the client
    - HTTP closes the connection.
- 2 types of HTTP Connections: Non-persistent HTTP connection and Persistent HTTP connection

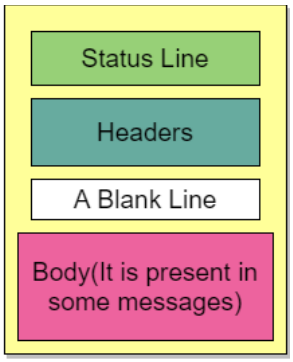
Non-persistent HTTP connection	Persistent HTTP connection
Non-persistent HTTP connection is one that is used for serving exactly one request and sending one response.	Persistent HTTP connection is one that can be used for serving multiple requests.
HTTP server closes the TCP connection automatically after sending a HTTP response.	HTTP server closes the TCP connection only when it is not used for a certain configurable amount of time.
A new separate TCP connection is used for each object.	A single TCP connection is used for sending multiple objects one after the other.
HTTP 1.0 supports non-persistent connections by default.	HTTP 1.1 supports persistent connections by default.
Example- Suppose a request has been made for a HTML page that contains 10 images (called objects). Then, With non-persistent connection, all the 11 objects (1 page + 10 images) will be sent one by one. For getting each object, a new separate connection will be opened and used.	**Example-** Suppose a request has been made for a HTML page that contains 10 images (called objects). Then, With persistent connection, all the 11 objects (1 page + 10 images) will be sent one after the other using a single TCP connection.

1.2.5.1 HTTP Message

- HTTP message is the information passed between the client and server.
- Two types
  - Requests (Client to server)
  - Responses (Server to client)



Request Message

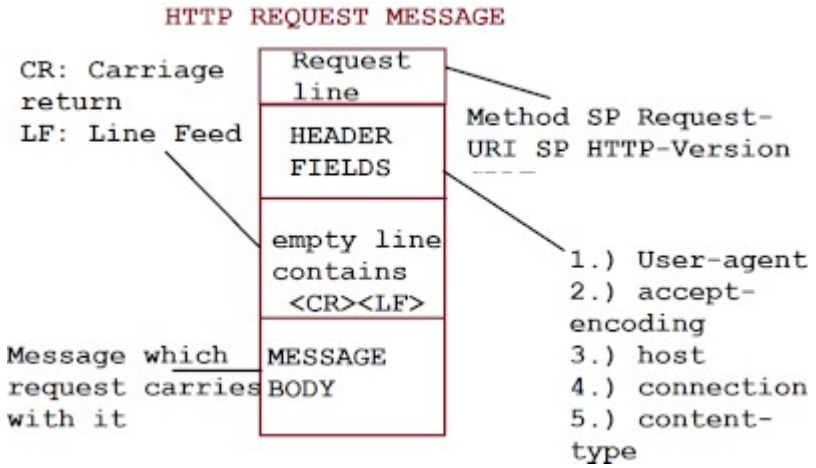


Response Message

1.2.5.2 HTTP Request Message

HTTP Request Message fields

- Request line comprising three parts, separated by spaces
- Headers
  - General header
  - Request header or Response header
  - Entity header
- Blank line
- Entity body



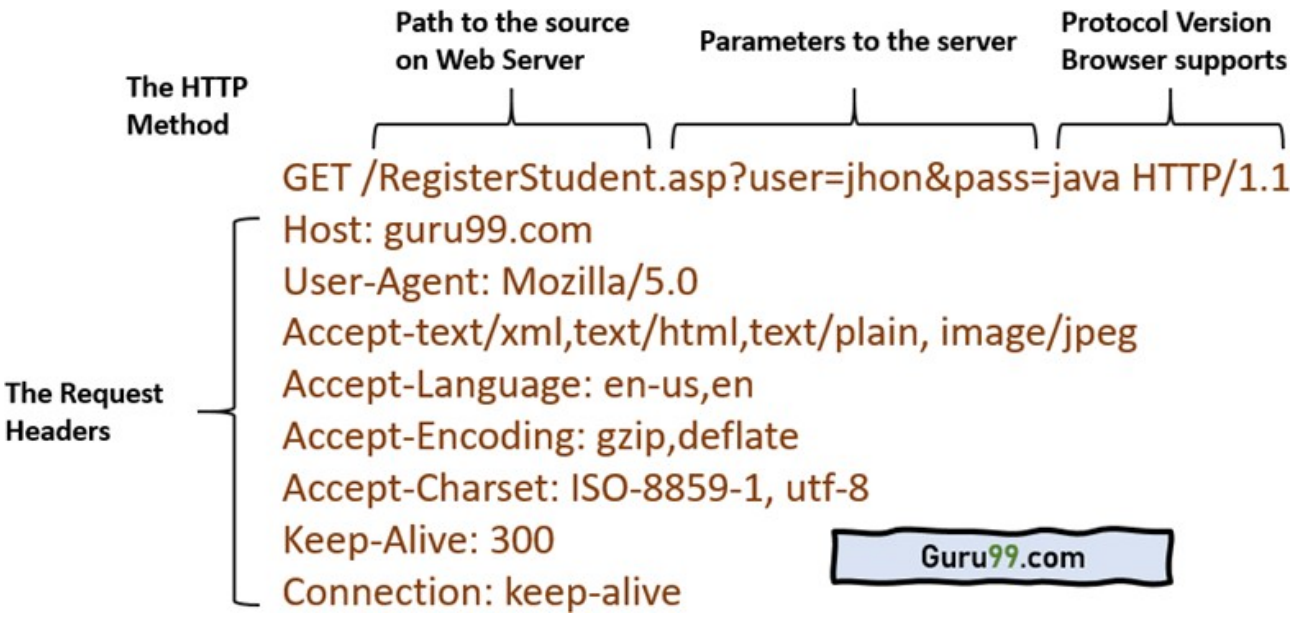
Request message examples

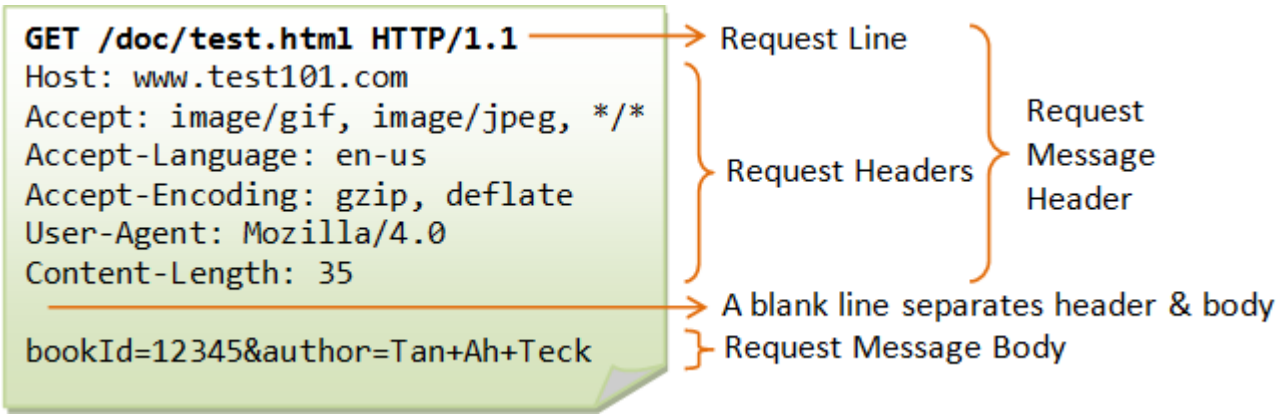
```
POST / HTTP/1.1
Host: localhost:8000
User-Agent: Mozilla/5.0 (Macintosh;… )… Firefox/51.0
Accept: text/html,application/xhtml+xml,…,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data; boundary=-12656974
Content-Length: 345

-12656974
(more data)
```

Request headers

General headers

Representation headers





Request Line

- Has three parts, with a single space used to separate adjacent parts
  - A method name
  - Request-URI (Uniform Resource Identifier) – the identifier of the resource,
  - The version of HTTP being used
- We consider each of the parts of the request line, in reverse order

1.2.5.2.1 HTTP Version

- Reflects the HTTP version in user
- Always takes the form "HTTP/x.x", uppercase
  - For current version of HTTP, HTTP/1.1

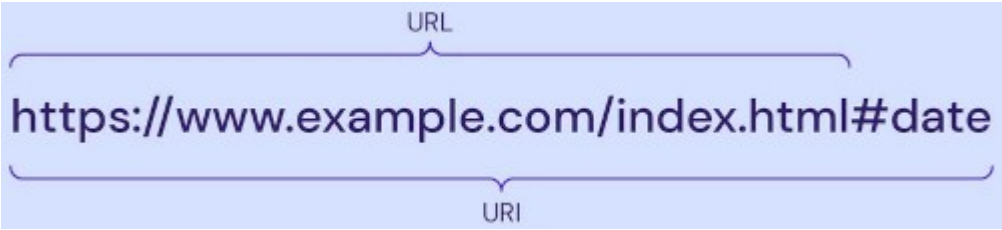
1.2.5.2.2 Request-URI

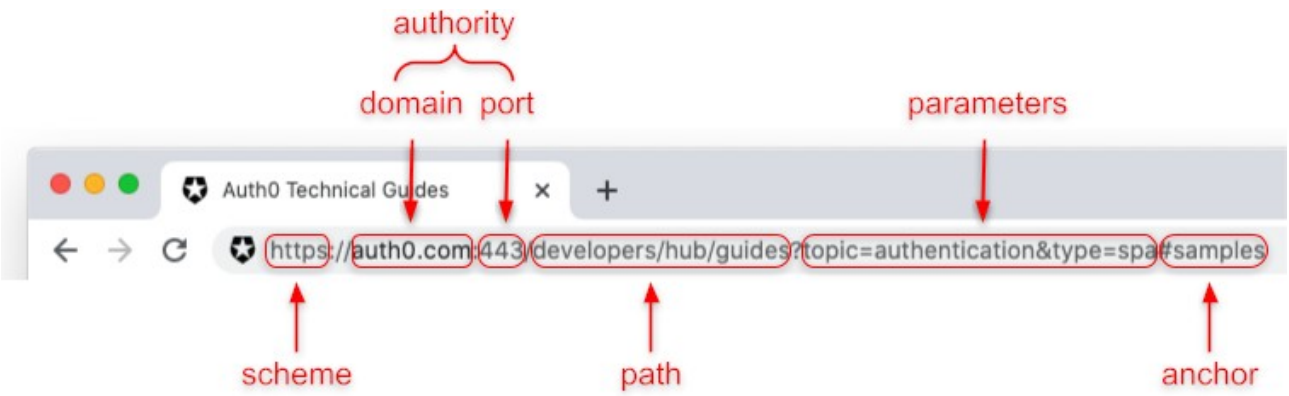
- Usually the local path of the requested resource, in which case the URI is also a URL (Uniform Resource Locator)
- Example of URI that is not a URL
  - URN (Uniform Resource Name). A URN is designed to be a unique name for a resource rather than specifying a location

URI vs URL

- Both URLs and URIs are ways of finding something online
- A URI
  - Is an identifier that is intended to be associated with a particular resource, (e.g., a web page, a graphics image, etc.) on the Web
  - Can be a name by itself, OR a name with a protocol
- URL (Uniform Resource Locator)
  - A URL is a subtype of URI with both a resource and a protocol
    - Here, the URI represents the location of a resource on the Web
    - The scheme (or protocol) appears before the colon (:)
    - The resource, which depends on the scheme, follows the colon
    - Web addresses, for the most part, use the HTTP scheme
- Note: All URLs are URIs, but not all URIs are URLs

Examples





Some other URI schemes that mark the URI as a URL

Scheme Name	Example URL	Type of Resource
`ftp`	`ftp://ftp.example.org/pub/afile.txt`	File located on FTP server
`telnet`	`telnet://host.example.org/`	Telnet server
`mailto`	`mailto:someone@example.org`	Mailbox
`https`	`https://secure.example.org/sec.txt`	Resource on web server supporting encrypted communication
`file`	`file:///C:/temp/localFile.txt`	File accessible from machine processing this URL

Uniform Resource Name (URN)

- URNs are
  - Defined namespaces assigned by a namespace authority, so they remain unique and persistent over time even after the resource they are attached to moves or ceases to exist
  - Globally unique persistent identifiers assigned within defined namespaces so they will be available for a long period of time, even after the resource which they identify ceases to exist or becomes unavailable
- Unlike a URL which becomes invalid when data is moved to a different location, the URN remains constant when objects and resources move
- URNs cannot be used to directly locate an item and need not be resolvable, as they are simply templates that another parser may use to find an item

Structure of a URN

- URI syntax is used
- Consists of three colon-separated parts

  urn:<nid>:<nss>
  - **urn**: this is scheme, which is always **urn** for a URN-type URI
  - **nid**: Namespace identifier; can include letters, digits and dashes
  - **nss**: Namespace-specific string; its representation depends on the specified namespace

- To ensure the global uniqueness of URN namespaces, their persistent identifiers (NIDs) are required to be registered with the Internet Assigned Numbers Authority.

URN example

- The ISBN of an edition of the book War and Peace: 0-1404-4417-3
  - This is the only book worldwide with this number
  - Associated URN for this book can be written as follows:
    urn:ISBN:0-1404-4417-3

1.2.5.2.3 Request Method Name

Standard HTTP methods

Method	Requests server to . . .
GET	return the resource specified by the Request-URI as the body of a response message.
POST	pass the body of this request message on as data to be processed by the resource specified by the Request-URI.
HEAD	return the same HTTP header fields that would be returned if a GET method were used, but not return the message body that would be returned to a GET (this provides information about a resource without the communication overhead of transmitting the body of the response, which may be quite large).
OPTIONS	return (in Allow header field) a list of HTTP methods that may be used to access the resource specified by the Request-URI.
PUT	store the body of this message on the server and assign the specified Request-URI to the data stored so that future GET request messages containing this Request-URI will receive this data in their response messages.
DELETE	respond to future HTTP request messages that contain the specified Request-URI with a response indicating that there is no resource associated with this Request-URI.
TRACE	return a copy of the complete HTTP request message, including start line, header fields, and body, received by the server. Used primarily for test purposes.

- Note: Method names are always uppercase
- GET and POST are the most frequently used methods

GET method

- This is the primary HTTP method
- It is the method that is
  - Used when you type a URL into the location bar of your browser
  - Used by default when you click on a link in a document displayed in your browser and when the browser downloads images for display within an HTML document
- Used for queries that can be safely repeated
- Used to retrieve information from a specified URI and is assumed to be a safe, repeatable operation by browsers, caches and other HTTP aware components
- Operations have no side effects and GET requests can be re-issued
  - For example, displaying the balance of a bank account has no effect on the account and can be safely repeated

- Most browsers will allow a user to refresh a page that resulted from a GET, without displaying any kind of warning
- Proxies may automatically retry GET requests if they encounter a temporary network connection problem
- GET requests supply data in the form of parameters encoded in the URI (known as a Query String)
- Cannot be used for uploading files or other operations that require large amounts of data to be sent to the server

POST method

- Typically used to send information collected from a form displayed within a browser to the web server
- Used for operations that may have side effects and cannot be safely repeated
  - For example, transferring money from one bank account to another has side effects and should not be repeated without explicit approval by the user

1.2.5.3 Header fields and MIME Types

- Host header field (e.g., google.com, example.com) is required in every HTTP/1.1 request message
- HTTP/1.1 also defines a number of other header fields, several of which are commonly used by modern browsers
- Each header field begins with a field name, such as Host, followed by a colon and then a field value
- Example of an HTTP request sent by a browser consisting of a start line, 10 header fields, and a short message body:

```
POST /servlet/EchoHttpRequest HTTP/1.1
host: www.example.org:56789
user-agent: Mozilla/5.0 (Windows; U; Windows NT 5.1; en-US; rv:1.4)
  Gecko/20030624
accept: text/xml,application/xml,application/xhtml+xml,
  text/html;q=0.9,text/plain;q=0.8,video/x-mng,image/png,image/jpeg,
  image/gif;q=0.2,*/*;q=0.1
accept-language: en-us,en;q=0.5
accept-encoding: gzip,deflate
accept-charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7
connection: keep-alive
keep-alive: 300
content-type: application/x-www-form-urlencoded
content-length: 13

doit=Click+me
```

Some notes on header fields

- Header names are not case sensitive
- A header field value may wrap onto several lines by preceding each continuation line with one or more spaces or tabs
  - This also means that a header field name must begin at the first character of a line, with no preceding white space.

Use of MIME types in header field values

- MIME: Multipurpose Internet Mail Extensions
- Refers to a standard that can be used to pass a variety of types of information, including graphics and applications, through e-mail and through other Internet message protocols
- The content of a MIME message is specified using a two-part, case-insensitive string which, in web applications, is known as the content type of the message
- Examples of standard MIME content-type strings: text/html and image/jpeg
  - The substring preceding the slash in these strings is the top-level type (see table below)

Standard top level MIME content types

Top-level Content Type	Document Content
application	Data that does not fit within another content type and that is intended to be processed by application software, or that is itself an executable binary.
audio	Audio data. Subtype defines audio format.
image	Image data, typically static. Subtype defines image format. Requires appropriate software and hardware in order to be displayed.
message	Another document that represents a MIME-style message. For example, following an HTTP TRACE request message to a server, the server sends a response with a body that is a copy of the HTTP request. The value of the Content-Type header field in the response is message/http.
model	Structured data, generally numeric, representing physical or behavioral models.
multipart	Multiple entities, each with its own header and body.
text	Displayable as text. That is, a human can read this document without the need for special software, although it may be easier to read with the assistance of other software.
video	Animated images, possibly with synchronized sound.

- The substring following the slash, called the subtype, specifies the particular type of content relative to the top-level type (see table below for examples)

Some Common MIME Content Types

MIME Type	Description
text/html	HTML document
image/gif	Image represented using Graphics Interchange Format (GIF)
image/jpeg	Image represented using Joint Picture Expert Group (JPEG) format
text/plain	Human-readable text with no embedded formatting information
application/octet-stream	Arbitrary binary data (may be executable)
application/x-www-form-urlencoded	Data sent from a web form to a web server for processing

- Private (i.e., unregistered) MIME top-level types and subtypes may be used
  - Indicated by an “x-” (or “X-”) prefix

Quality values

- Many header field values use *quality values* to indicate preferences
- Quality value: Specified by a string of the form
  ;q=num, where num is a decimal number between 0 and 1, with a higher number representing greater preference
  - Each quality value applies to all of the comma-separated field values preceding it back to the next earlier quality value
  - In the example HTTP request above
    - According to the Accept header field the browser prefers text/xml (quality value 0.9) over image/jpeg (quality value 0.2)

The use of the * character in a header field value

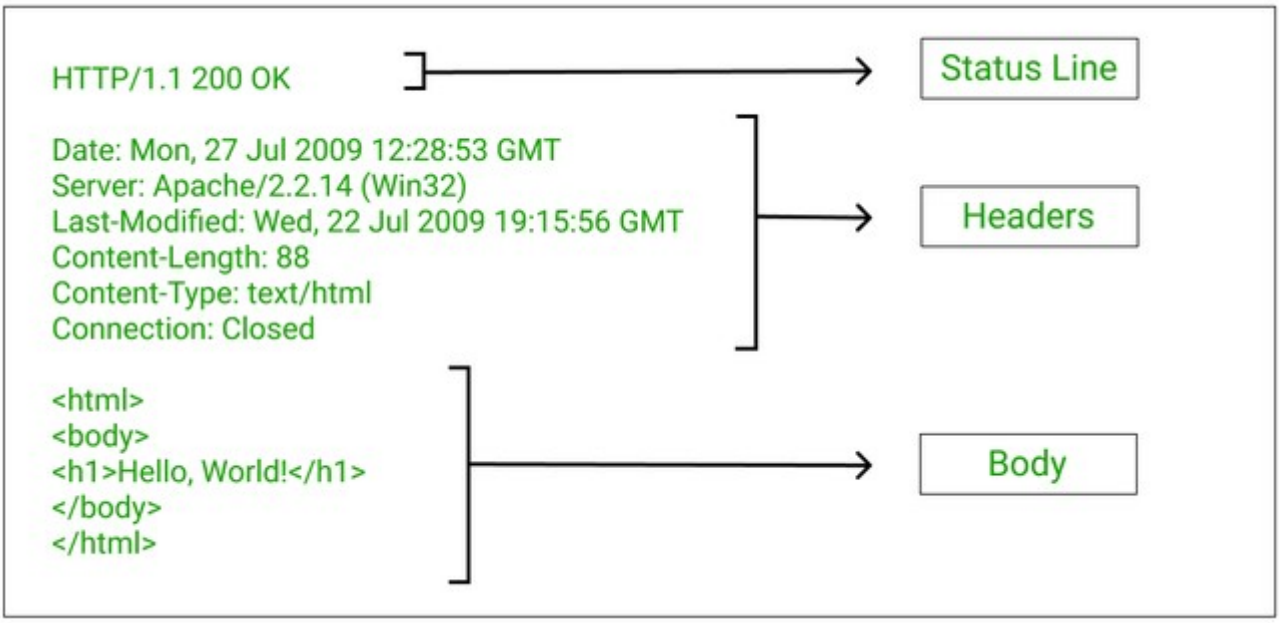
- * is used as a wildcard character
- Example: the string */* in the Accept header field value represents all possible MIME types

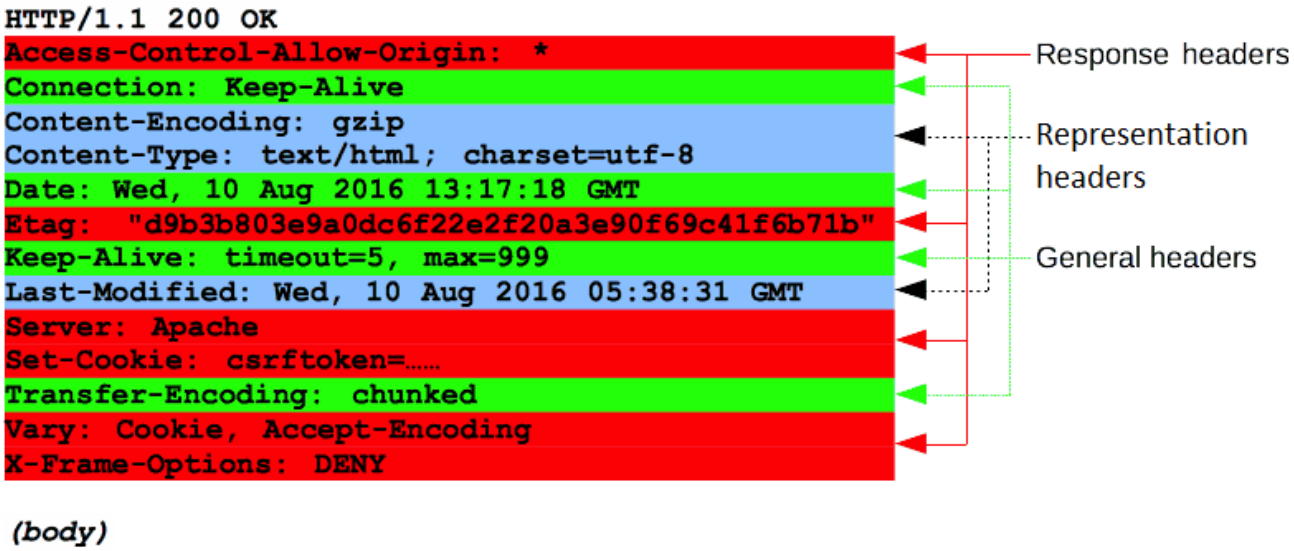
Common HTTP/1.1 Request Header fields

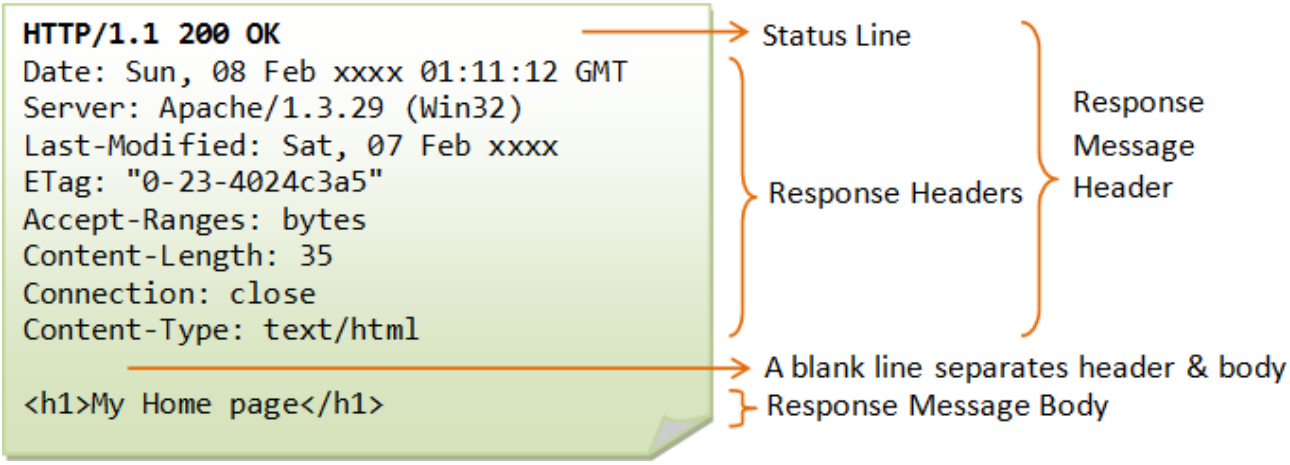
Field Name	Use
Host	Specify *authority* portion of URL (host plus port number; see Section 1.6.2). Used to support *virtual hosting* (running separate web servers for multiple fully qualified domain names sharing a single IP address).
User-Agent	A string identifying the browser or other software that is sending the request.
Accept	MIME types of documents that are acceptable as the body of the response, possibly with indication of preference ranking. If the server can return a document according to one of several formats, it should use a format that has the highest possible preference rating in this header.
Accept-Language	Specifies preferred language(s) for the response body. A server may have several translations of a document, and among these should return the one that has the highest preference rating in this header field. For complete information on registered language tags, see [RFC-3066] and [ISO-639-2].
Accept-Encoding	Specifies preferred encoding(s) for the response body. For example, if a server wishes to send a compressed document (to reduce transmission time), it may only use one of the types of compression specified in this header field.
Accept-Charset	Allows the client to express preferences to a server that can return a document using various character sets (see Section 1.5.4).
Connection	Indicates whether or not the client would like the TCP connection kept open after the response is sent. Typical values are `keep-alive` if connection should be kept open (the default behavior for servers/clients compatible with HTTP/1.1), and `close` if not.
Keep-Alive	Number of seconds TCP connection should be kept open.
Content-Type	The MIME type of the document contained in the message body, if one is present. If this field is present in a request message, it normally has the value shown in the example, application/x-www-form-urlencoded.
Content-Length	Number of bytes of data in the message body, if one is present.
Referer	The URI of the resource from which the browser obtained the Request-URI value for this HTTP request. For example, if the user clicks on a hyperlink in a web page, causing an HTTP request to be sent to a server, the URI of the web page containing the hyperlink will be sent in the Referer field of the request. This field is not present if the HTTP request was generated by the user entering a URI in the browser's Location bar.

1.2.5.4 HTTP Response Message

Response message examples







HTTP response message comprises

- Status Line comprising three parts, separated by spaces
  - The HTTP version
  - A response status code that gives the result of the request, and
  - An English reason phrase describing the status code
- Headers (one or more)
  - General header
  - Request header or Response header
  - Entity header
- Blank line
- Message body (optional)

We use the following response message in the examples that follow.

```
HTTP/1.1 200 OK
Date: Thu, 09 Oct 2003 20:30:49 GMT
Server: Apache/1.3.27 (Unix) (Red-Hat/Linux)
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Content-Length: 438
Connection: close
Content-Type: text/html

<HTML>
<HEAD>
<TITLE>Example Web Page</TITLE>
</HEAD>
<BODY>
<p>You have reached this web page by typing &quot;example.com&quot;,
&quot;example.net&quot;,
or &quot;example.org&quot; into your web browser.</p>
<p>These domain names are reserved for use in documentation and are
not available for registration. See
<a href="http://www.rfc-editor.org/rfc/rfc2606.txt">RFC 2606</a>,
Section 3.</p>
</BODY>
</HTML>
```

1.2.5.4.1 HTTP Response Status Line

- **HTTP/1.1 200 OK** in this example
- Consists of three fields
  - The HTTP version used by the server software
  - A numeric status code indicating the type of response
    - A 3-digit decimal (200 in this example)
    - The first digit represents the general class of status code (see other classes in the table below)

HTTP/1.1 Status Code Classes (first Digit of Status Code)

Digit	Class	Standard Use
1	Informational	Provides information to client before request processing has been completed.
2	Success	Request has been successfully processed.
3	Redirection	Client needs to use a different resource to fulfill request.
4	Client Error	Client's request is not valid.
5	Server Error	An error occurred during server processing of a valid client request.

- The last two digits of a status code define the specific status within the specified class (see common HTTP/1.1 status codes below)

Common HTTP/1.1 status codes

Status Code	Recommended Reason Phrase	Usual Meaning
200	OK	Request processed normally.
301	Moved Permanently	URI for the requested resource has changed. All future requests should be made to URI contained in the Location header field of the response. Most browsers will automatically send a second request to the new URI and display the second response.
307	Temporary Redirect	URI for the requested resource has changed at least temporarily. This request should be fulfilled by making a second request to URI contained in the Location header field of the response. Most browsers will automatically send a second request to the new URI and display the second response.
401	Unauthorized	The resource is password protected, and the user has not yet supplied a valid password.
403	Forbidden	The resource is present on the server but is read protected (often an error on the part of the server administrator, but may be intentional).
404	Not Found	No resource corresponding to the given Request-URI was found at this server.
500	Internal Server Error	Server software detected an internal failure.

- A text string (the reason phrase) (OK in this example) that presents the information represented by the numeric status code in human-readable form.
- This particular status code indicates that no errors were detected by the server. The body of a response having this status code should contain the resource requested by the client.

1.2.5.4.2 Response Header fields

- Some HTTP request header fields are also valid in response messages
  - e.g., Connection, Content-Type, and Content-Length
- The Content-Type of a response can be any one of the MIME type values specified by the Accept header field of the corresponding request

Some other common response header fields

Field Name	Use
Date	Time at which response was generated. Used for cache control (see Section 1.5.3). This field must be supplied by the server.
Server	Information identifying the server software generating this response.
Last-Modified	Time at which the resource returned by this request was last modified. Can be used to determine whether cached copy of a resource is valid or not (see Section 1.5.3).
Expires	Time after which the client should check with the server before retrieving the returned resource from the client's cache (see Section 1.5.3).
ETag	A hash code of the resource returned. If the resource remains unchanged on subsequent requests, then the ETag value will also remain unchanged; otherwise, the ETag value will change. Used for cache control (see Section 1.5.3).
Accept-Ranges	Clients can request that only a portion (*range*) of a resource be returned by using the Range header field. This might be used if the resource is, say, a large PDF file and only a single page is currently needed. Accept-Ranges specifies the units that may be used by the client in a range request, or none if range requests are not accepted by this server for this resource.
Location	Used in responses with redirect status code to specify new URI for the requested resource.

1.2.5.4.3 Cache Control

Read pages 20-21 of assigned readings

1.2.5.4.4 Character Sets

Read pages 21-22 of assigned readings

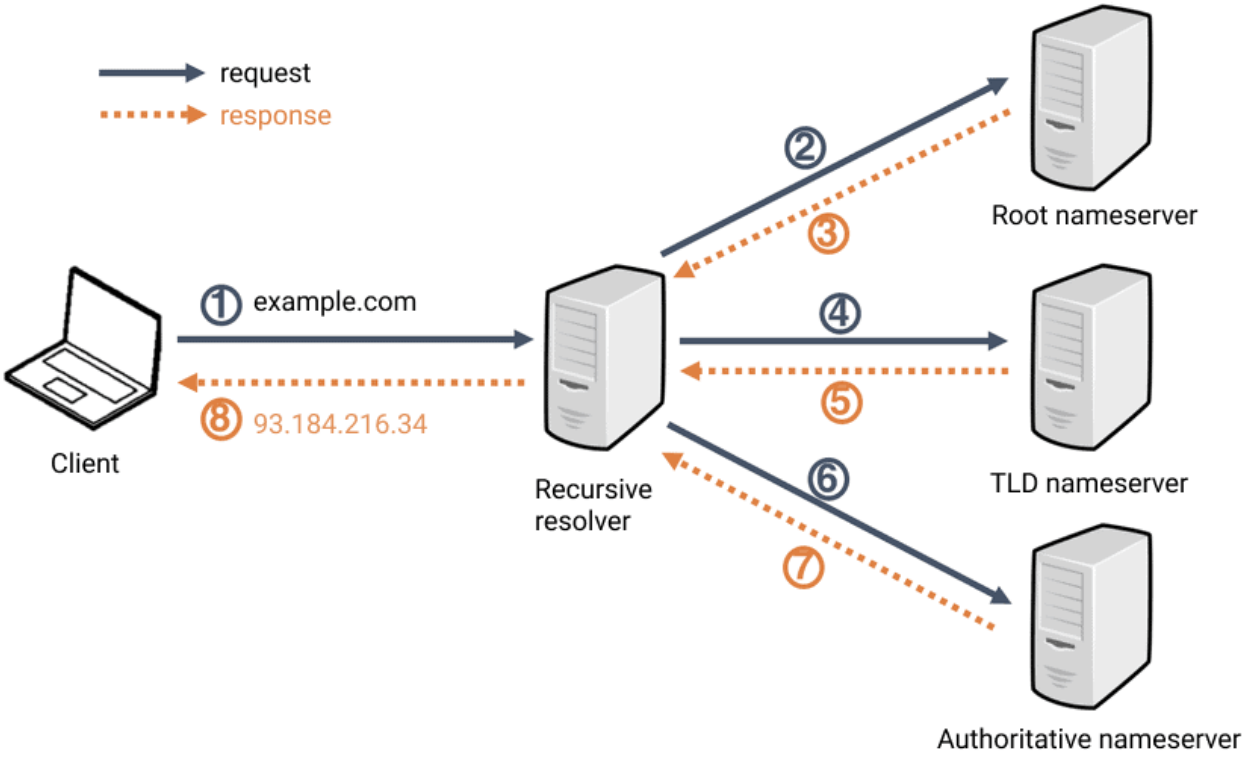
1.2.6 DNS (Domain Name Service)

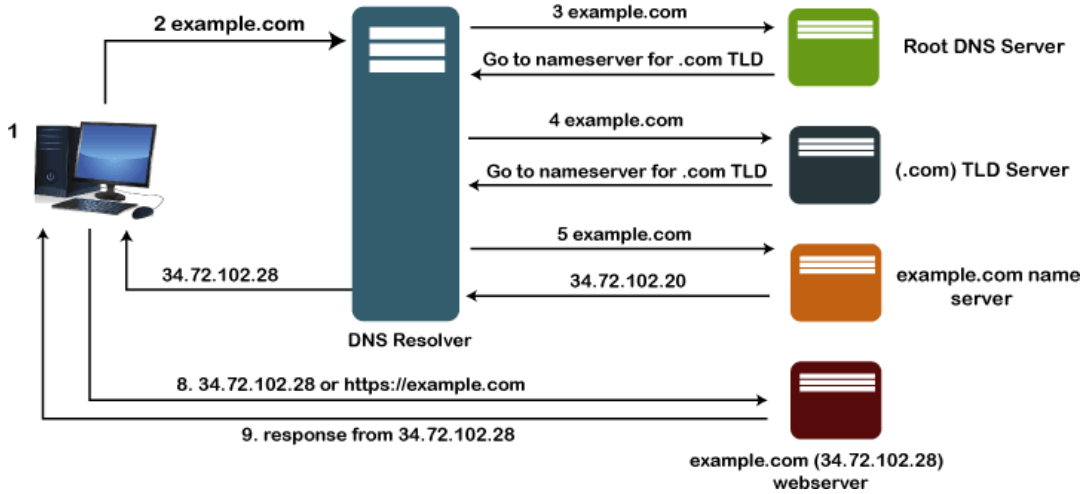
- DNS is a host name to IP Address translation service
- Converts the names we type in our web browser address bar to the IP Address of web servers hosting those sites

- Need for DNS
  - IP Addresses are not static and may change dynamically
    - So, a mapping is required which maps the domain names to the IP Addresses of their web servers
  - IP Addresses are a complex series of numbers
    - So, it is difficult for the user to remember IP Addresses directly while it is easy to remember names
    - The names need to be resolved to the corresponding IP addresses

Steps involved in domain name resolution

1. A user program sends a name query to a library procedure called the DNS resolver
2. Resolver looks up the local domain name cache for a match
   - If a match is found, it sends the corresponding IP Address back
   - If no match is found, it sends a query to the local DNS server
3. DNS server looks up the name
   - If a match is found, it returns the corresponding IP Address to the resolver
   - If no match is found, the local DNS server sends a query to a higher level DNS server
   - This process is continued until a result (an IP address) is returned to the DNS resolver
4. The DNS resolver now has the IP address that the user needs
   - The resolver returns that value to the web browser (and possibly to the application that is running on the web browser)
   - The DNS resolver also caches (stores) the IP address for example.com for an amount of time (time to live) that you specify so that it can respond more quickly the next time someone browses to example.com
5. The web browser can now send a request for www.example.com to the IP address that it got from the DNS resolver for the content of www.example.com
6. The web server or other resource at the IP address returns the web page for www.example.com to the web browser, and the web browser displays the page

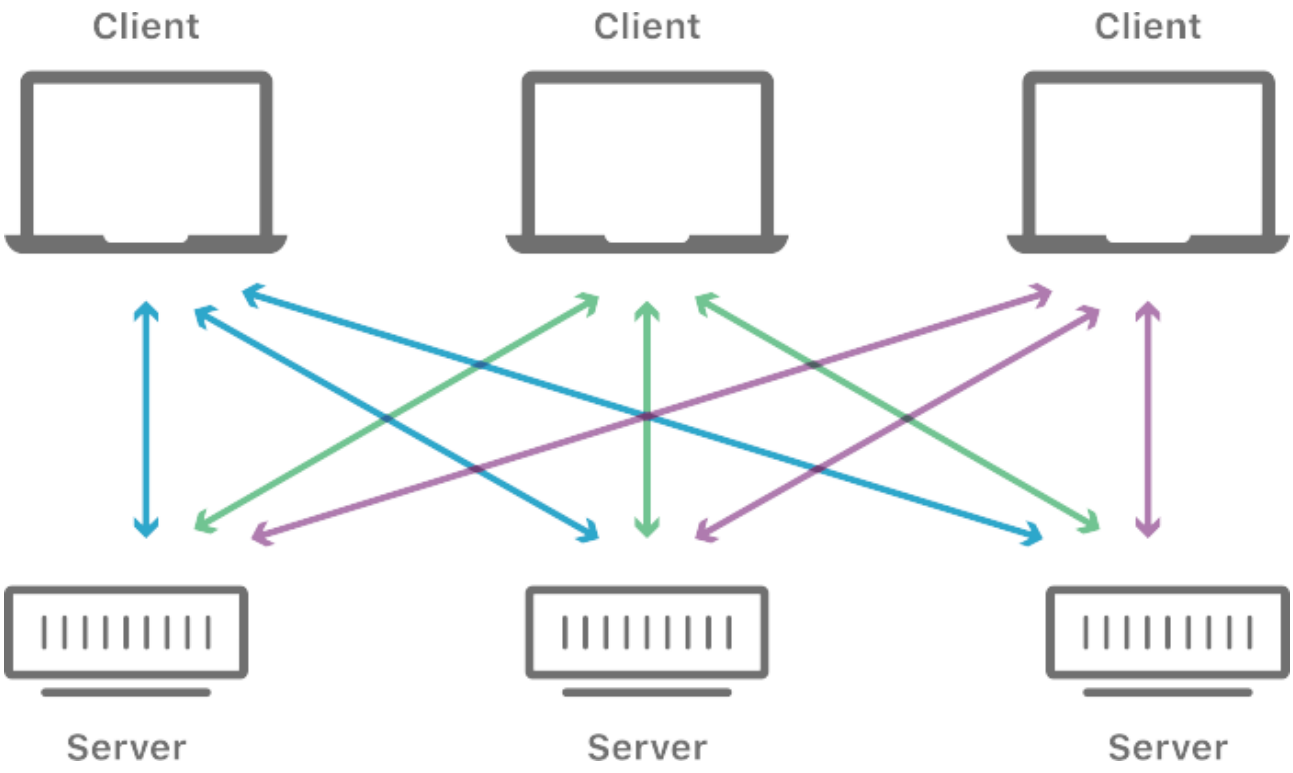

request
response
Root nameserver
Client
① example.com
②
③
④
⑤
⑥
⑦
⑧ 93.184.216.34
Recursive resolver
TLD nameserver
Authoritative nameserver


2 example.com
3 example.com
Go to nameserver for .com TLD
Root DNS Server
1
4 example.com
Go to nameserver for .com TLD
(.com) TLD Server
5 example.com
34.72.102.28
34.72.102.20
example.com name server
DNS Resolver
8. 34.72.102.28 or https://example.com
9. response from 34.72.102.28
example.com (34.72.102.28) webserver

1.3 Client-server model

1.3.1 Introduction

- Basis of much of internet applications
  - User devices (i.e., clients – laptops, smartphones, and desktop computers) communicate via a network with centrally located servers to get the data they need, instead of communicating with each other
  - Client devices send requests to the servers for webpages or applications, and the servers serve up responses
- Servers can serve multiple client devices at once, and each client device sends requests to multiple servers in the course of accessing and browsing the Internet.



1.3.2 Client-side in web development

- Everything in a client-server application that is displayed or takes place on the client device
  - This includes what the user sees, such as text, images, and the rest of the UI, along with any actions that an application performs within the user's browser
- In web applications
  - Markup languages like HTML as well as CSS are interpreted by the browser on the client side
  - Client-side processes are almost always written in JavaScript
- Client-side also commonly referred to as the frontend in applications that have a user interface

1.3.3 Server-side

- Everything in a client-server application that happens on the server
- In the past, nearly all business logic (dynamic webpages, interacting with databases, etc) ran on the server side
- The problem with hosting all of these processes on the server side
  - Each request involving one of them has to travel all the way from the client to the server, every time; this introduces a great deal of latency
  - Server overloaded
  - For these reasons, contemporary applications run more code on the client side
- Server-side *processes* also commonly referred to as the backend in applications that have a user interface

1.3.4 Client-side Scripting vs Server-side Scripting

Client-side scripting

- Means running scripts on the client device, usually within a browser
- In web applications
  - Markup languages like HTML as well as CSS are interpreted by the browser on the client side
  - Client-side processes are almost always written in JavaScript because JavaScript is universally supported
    - Other scripting languages can only be used if the user's browser supports them.

Server-side scripts

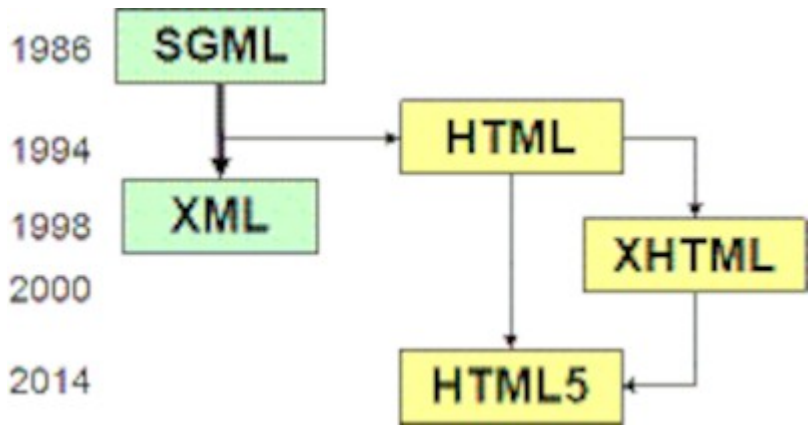
- Run on the server instead of the client, often in order to deliver dynamic content to webpages in response to user actions
- Server-side scripts don't have to be written in JavaScript, since the server may support a variety of languages.

1.4 Markup Languages

- Markup language
  - SGML
  - HTML
  - XML
  - XHTML
- HTML Tutorial

1.4.1 Markup language

- A set of rules governing what markup information may be included in a document and how it is combined with the content of the document in a way to facilitate use by humans and computer programs
- A set of characters or symbols that define a document’s logical structure or how a document should be printed or displayed
- The idea and terminology evolved from the "marking up" of paper manuscripts (i.e., the revision instructions by editors), which is traditionally written with a red pen or blue pencil on authors' manuscripts
- SGML, XHTML, HTML, and XML are all markup languages



1.4.1.1 SGML (Standard Generalized Markup Language)

- The international standard for all markup languages for data exchange and storage
- Provides a way to define markup languages and sets the standard for their form
  - States what some languages can or cannot do, what elements must be included, such as tags, and the basic structure of the language
- Descriptive, rather than procedural
  - i.e., the same document can be processed by different systems; each system applies different processing instructions to relevant sections
- Was the first language to implement the Document Type Definition (DTD)
  - DTD formally defines the document by its components and structure
  - A document that conforms to the structure of a DTD is said to be valid
  - Hence, documents of the same type can then be verified and processed uniformly

1.4.1.2 HTML (HyperText Markup Language)

- An application of SGML
- The predominant language used in website design
- Structures the page for a browser
  - With HTML, you can embed images, create page sections, establish fonts, and direct the flow of the page
  - Using HTML, you can add other functions to a website via scripting languages such as JavaScript

1.4.1.3 XML (Extensible Markup Language)

- A generalized format for representing structured information, especially for the web
- A subset of SGML, which gives it rights that an application, such as HTML, does not have
  - HTML is limited to design and does not have subsets or applications
  - XML is a generic markup language suitable for representing arbitrary data, while HTML is a specific markup language suitable only for representing web pages
- Extensible
  - You can define not only your tags but also their order, relationships among them, and the way they are processed and displayed
- Can define applications of its own
  - Example: Resource Description Format (RDF) is an application of XML
- Designed to describe data and it has strict syntax rules

    - The XML specification requires that XML processors not accept XML documents with any errors
    - Notes
        - Whereas HTML describes formatting, XML describes content itself
        - XML is for defining your own data format. If you wish to pass data between two systems, XML is often the way to do it
- Valid and Well-formed XML-based
    - An XML document that conforms to the structure of a DTD is said to be valid
    - An XML document that uses tags that conform to the standard XML specifications is said to be well-formed

1.4.1.4 XHTML (Extensible Hypertext Markup Language)

- An extension of HTML that is based on XML and is designed to work with XML-based applications
- HTML and XHTML are both subsets only of SGML, except that XHTML has additional specifications so that it also validates as XML
- Primary purpose is to remove a lot of the ambiguity in HTML
    - Most web pages have some kind of syntax error
    - But browsers tend to ignore web page errors and go ahead to render the pages in spite of the errors
    - XHTML was developed to make HTML more extensible and flexible to work with other data formats by imposing XML structure, and recommendations were made for browsers to stop processing web pages with syntax errors
        - This idea was not universally popular (especially with the higher error rates - which translates to error messages)
        - Web authors basically ignored incorporating the stricter XHTML in their design, and today, HTML is the universally adopted language for web page markup.

1.4.2 Origins and Evolution of HTML (Hypertext Markup Language)

- Developed for the delivery of hypertext on the WWW
- HTML was designed using SGML
- Original intent of HTML
    - To specify document structure at a higher and more abstract level, because HTML-specified documents had to be displayable on a variety of computer systems using different browsers
- Recent Versions
    - HTML 4.0 – 1997
        - Introduced many new features and deprecated many older features
    - HTML 4.01 – 1999
        - A clean-up of 4.0
        - Major problems of HTML 4.01
            - *Loose syntax rules*. Hence, many variations of document forms which may be interpreted differently by different browsers
            - Specification does not define how a user agent (e.g., browser) is to recover when erroneous code is encountered. Consequently, every browser uses its own variation of error recovery.
    - XHTML (Extensible HTML) 1.0 – 2000
        - Just 4.01 defined using XML, instead of SGML

        - XML specification requires that XML processors not accept XML documents with any errors
        - Provides complete rules stating what is and what is not syntactically acceptable
        - User agents (e.g., browsers) were required to halt interpretation of an XHTML document with any syntax error
            - But because this was a drastic response to the error-ignoring nature of HTML 4.01, the XHTML 1.0 included the option for XHTML documents to be served as HTML (thus allowing continuation of the practice of user agents ignoring syntactic errors)
    - XHTML 1.1 – 2001
        - Modularized 1.0
        - Dropped some features of XHTML 1.0, notably frames
        - Eliminated the XHTML loophole that allowed rendering of documents with syntactic errors
            - Hence, user agents were required to reject all syntactically incorrect documents. This is an example of "draconian error handling"
            - Notice tension here
                - The value of consistent and coherent syntax rules of XHTML (widely recognized and accepted and favoured by W3C) vs draconian error handling (not widely accepted, as it was generally agreed by developers and providers of markup documents that browser users should not be giving error messages due to the syntax errors found in documents they were attempting to view)
                - Hence, XHTML 1.1 browsers were allowed to continue to use forgiving HTML parsers
- XHTML 2.0
    - Tension highlighted above let to W3C opting for XHTML 2.0
    - A new organization, Web Hypertext Application Technology (WHAT) Working Group chose a different path and began working on the next version of HTML, which was to be based on HTML 4.01, rather thatn XHTML 1.1
- HTML 5.0 – 2009
    - Good progress made by the WHAT Working Group
        - WebForm 2.0, which extended HTML forms
        - Web Applications 1.0, which was a new version of HTML
        - An algorithm for user agent error handling
    - W3C dropped development work on XHTML 2.0 and adopted the HTML development
    - Web Applications 1.0 renamed HTML5

1.4.3 Comprehensive HTML tutorial and readings

- https://www.w3schools.com/html/
- Internet & World Wide Web: How to Program
    - Introduction to HTML5: Part 1 (Pages 69 to 107)
    - Introduction to HTML5: Part 2 (Pages 108 to 136)